

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Intelligent Multi-Agent and Expert Systems (AI401)
Semester VII

Experiment 1
Knowledge Representation - Semantic Network with Inheritance

Name Aditya Gaur

Roll Number U23AI095

Aim

To represent knowledge as a semantic network of (subject, relation, object) triples and answer queries using property inheritance along **is-a** links, including exceptions.

Theory

Before an expert system can reason, domain knowledge must be represented in a form that a program can process. A semantic network represents concepts as nodes and relationships as labelled, directed links. The **is-a** relation organizes concepts into a taxonomy, allowing properties of a general concept to be inherited by a more specific concept.

For example, the knowledge

shark is-a fish, fish is-a animal, animal has skin

allows the system to infer that a shark has skin through the inheritance chain **shark -> fish -> animal**. The representation must also support exceptions. The default fact **bird can fly** is overridden for a penguin by the more specific fact **penguin cannot fly**. Therefore, the query mechanism searches from the most specific concept upward and gives priority to local exception knowledge before inherited values. This is the default-with-exceptions behaviour described in the experiment and is an important foundation for later rule-based reasoning.

Algorithm

1. Store the knowledge network as a list of (subject, relation, object) triples.
2. For an inheritance chain, repeatedly follow an **is-a** link from the current node to its parent until no parent exists.
3. To answer **query(node, relation)**, traverse the chain from the most specific node upward. At every level, check first for a matching exception represented by **cannot**, and then check for a local value of the requested relation.
4. Return the first value found. If the entire chain is exhausted without a result, return **unknown**.

Procedure and Results

The implementation was executed in Python. The knowledge base, inheritance chains, query results, ostrich extension, frame representation, and multiple-inheritance behaviour were verified using the program output shown below.

1. Basic Semantic Network

The initial knowledge base contains the following triples:

Subject	Relation	Object
animal	has	skin
fish	is-a	animal
shark	is-a	fish
bird	can	fly
penguin	is-a	bird
penguin	cannot	fly

The observed inheritance chains and queries are important because they demonstrate both ordinary inheritance and exception handling:

- `shark -> fish -> animal` gives the inherited property `has skin`.
- `penguin -> bird` reaches the general rule `bird can fly`, but the local exception `penguin cannot fly` has priority.
- A query with no available fact or inherited property returns `unknown`.

```

PS C:\Users\adity\Desktop\Sem7> & "C:\Program Files\Python313\python.exe" c:/Users/adity/Desktop/Sem7/IMAESLab/lab01/LAB_01_U23AI095.py
=====
AI401 - EXPERIMENT 1
Knowledge Representation - Semantic Network with Inheritance
=====

--- BASIC SEMANTIC NETWORK ---
KNOWLEDGE BASE (TRIPLES)
('animal', 'has', 'skin')
('fish', 'is-a', 'animal')
('shark', 'is-a', 'fish')
('bird', 'can', 'fly')
('penguin', 'is-a', 'bird')
('penguin', 'cannot', 'fly')

Inheritance chain for shark:
shark -> fish -> animal

Inheritance chain for penguin:
penguin -> bird

Query results:
query('shark', 'has') = 'has skin'
query('shark', 'can') = 'unknown'
query('penguin', 'can') = 'cannot fly'
query('bird', 'can') = 'can fly'
query('lion', 'can') = 'unknown'

```

Figure 1: Basic semantic network: knowledge base, inheritance chains, and query results.

Interpretation of the Basic Results

The program returns:

Query	Result
query('shark','has')	has skin
query('shark','can')	unknown
query('penguin','can')	cannot fly
query('bird','can')	can fly
query('lion','can')	unknown

The most important result is `query('penguin','can') = 'cannot fly'`. Although the general parent concept `bird` has the property `can fly`, the more specific penguin node contains an explicit exception. The algorithm checks that exception before accepting the inherited value, so the specific knowledge wins.

2. Exercise 1 - Ostrich Extension

The network is extended with an ostrich as follows:

```
ostrich is-a bird,  ostrich cannot fly,  ostrich can run_fast.
```

This demonstrates that a child concept can inherit a general property while overriding it locally and also defining an additional property of its own. The verified results are:

- `query('ostrich','can')` returns `cannot fly`.
- `query('ostrich','can','run_fast')` returns `can run_fast`.
- `query('ostrich','can','fly')` returns `cannot fly`.

```
--- EXERCISE 1: OSTRICH ---
Inheritance chain for ostrich:
ostrich -> bird
query('ostrich', 'can') = 'cannot fly'
query('ostrich', 'can', 'run_fast') = 'can run_fast'
query('ostrich', 'can', 'fly') = 'cannot fly'
query('ostrich', 'has') = 'unknown'
```

Figure 2: Exercise 1: ostrich inheritance, exception, and additional ability.

3. Exercise 2 - Frame Representation

The same knowledge is re-implemented using a Python dictionary for each concept. Each dictionary contains slots and fillers, while the `ako` (a-kind-of) slot represents inheritance. For example:

Concept	Frame contents
animal	{ <code>'skin': 'present'</code> }
fish	{ <code>'ako': 'animal'</code> }
shark	{ <code>'ako': 'fish'</code> }
bird	{ <code>'can': 'fly'</code> }
penguin	{ <code>'ako': 'bird', 'cannot': 'fly'</code> }
ostrich	{ <code>'ako': 'bird', 'cannot': 'fly', 'can': 'run_fast'</code> }

The frame lookup follows the **ako** chain and uses the same exception-before-inheritance rule. Thus, frames provide the same default-with-exceptions behaviour using slots and fillers instead of explicit triples.

```
--- EXERCISE 2: FRAMES ---  
FRAMES  
  animal: {'skin': 'present'}  
  fish: {'ako': 'animal'}  
  shark: {'ako': 'fish'}  
  bird: {'can': 'fly'}  
  penguin: {'ako': 'bird', 'cannot': 'fly'}  
  ostrich: {'ako': 'bird', 'cannot': 'fly', 'can': 'run_fast'}  
  
Frame inheritance chain for penguin:  
  penguin -> bird  
  
Frame query results:  
  lookup('shark', 'skin') = 'skin present'  
  lookup('penguin', 'can') = 'cannot fly'  
  lookup('bird', 'can') = 'can fly'  
  lookup('fish', 'skin') = 'skin present'  
  lookup('ostrich', 'can', 'run_fast') = 'can run_fast'  
  lookup('ostrich', 'can', 'fly') = 'cannot fly'
```

Figure 3: Exercise 2: frame representation and frame-based inheritance lookup.

4. Exercise 3 - Multiple Inheritance

The network is extended so that a concept may have multiple **is-a** parents. In the implementation, multiple parents are searched using breadth-first traversal from the most specific concept. This gives the following levels for **flying_fish**:

```
Level 0:  flying_fish  
Level 1:  bird, fish  
Level 2:  animal
```

The search order is justified as follows: all nodes at a nearer inheritance depth are considered before nodes at a more distant depth, so a property inherited from a direct parent is preferred to one that is only available from a farther ancestor. Within the same level, the stored parent order is retained to make the result deterministic.

The verified results include **flying_fish** can fly, inherited from the direct parent **bird**, and **flying_fish** has skin, inherited through **fish** -> **animal**. The penguin exception continues to override the bird default.

```
--- EXERCISE 3: MULTIPLE INHERITANCE ---
Inheritance levels for flying_fish:
  Level 0: ['flying_fish']
  Level 1: ['bird', 'fish']
  Level 2: ['animal']

Multiple-inheritance query results:
  query('flying_fish', 'can') = 'can fly'
  query('flying_fish', 'has') = 'has skin'
  query('penguin', 'can') = 'cannot fly'

ALL CORRECTNESS TESTS PASSED.
PS C:\Users\adity\Desktop\Sem7> □
```

Figure 4: Exercise 3: multiple inheritance levels and query results.

Observations

1. The query for `penguin can` returns `cannot fly` even though the parent concept `bird` has the default property `can fly`. This occurs because the more specific exception is checked before the inherited value.
2. Exceptions must be checked at every level of the inheritance chain before accepting inherited properties; otherwise a general default could incorrectly override more specific knowledge.
3. The result demonstrates non-monotonic default reasoning: adding the specific fact `penguin cannot fly` changes the conclusion that would otherwise be obtained from the general `bird` rule.
4. The ostrich extension shows that inheritance can coexist with exceptions and additional local properties.
5. The frame implementation produces the same reasoning behaviour using an `ako` inheritance slot, showing that the same declarative knowledge can be expressed using different representations.
6. For multiple inheritance, breadth-first search gives a principled nearest-parent-first order, while preserving parent insertion order within the same level for deterministic results.